

FuzzySimulated — Plataforma de Inferência Fuzzy

Sistema de Controle Fuzzy para Avaliação de Risco em Cibersegurança

Campo	Descrição
Disciplina	Inteligência Artificial e Computacional (0700M8)
Professor	Prof. Daniel Leal Souza
Turma	CC5MA / CC5NA
Equipe	Benjamin Yuji Suzuki (1 integrante)
Modalidade	Opção B (Produto) + Opção C-B (TSK) + Pontuação Extra PSO
Repositório	github.com/Benjamin-Yuji-Suzuki/FullStackEmRUST
Data	Junho/2026

1. Resumo

Este trabalho apresenta o **FuzzySimulated**, uma plataforma full-stack 100% Rust para construção, simulação e validação de sistemas de inferência fuzzy. O projeto aplica lógica fuzzy ao domínio de **cibersegurança**, modelando a avaliação de risco cibernético por meio de variáveis linguísticas como probabilidade de ataque, impacto financeiro e vulnerabilidade do sistema. São implementados dois motores de inferência — **Mamdani** (centroide) e **TSK** (média ponderada) — além de otimização de parâmetros via **PSO (Particle Swarm Optimization)** como pontuação extra. A plataforma oferece interface web reativa, geração de gráficos SVG, superfícies de controle 3D, diagnóstico explicativo e auditoria com undo via snapshots JSONB.

2. Introdução e Motivação

2.1 O Problema

A avaliação de risco em cibersegurança envolve múltiplas variáveis imprecisas: a probabilidade de um ataque, o impacto financeiro potencial e o nível de vulnerabilidade do sistema raramente podem ser expressos com precisão numérica. Um ataque pode ser "muito provável" ou "pouco impactante" — são julgamentos linguísticos, não medidas exatas. A lógica fuzzy é, portanto, naturalmente adequada para este domínio.

2.2 Por que Lógica Fuzzy?

Diferentemente da lógica binária tradicional (seguro vs. inseguro), a lógica fuzzy permite gradações: um sistema pode ter risco "médio-alto" ou "baixo-médio". Isso reflete melhor a realidade da cibersegurança, onde decisões são tomadas com base em informações parciais e avaliações qualitativas.

2.3 Objetivos

- Modelar um sistema fuzzy de avaliação de risco cibernético com 3 entradas e 1 saída
- Implementar motores Mamdani e TSK completos
- Disponibilizar interface web interativa para simulação
- Validar o sistema com cenários reais e fronteiriços
- Otimizar parâmetros automaticamente via PSO

2.4 Requisitos Funcionais e Não Funcionais

Requisitos Funcionais

ID	Descrição	UC
RF01	O sistema deve permitir criar, listar, editar e excluir sistemas fuzzy	UC01
RF02	O sistema deve permitir gerenciar variáveis (antecedentes/consequentes) e seus termos linguísticos com funções de pertinência	UC02
RF03	O sistema deve permitir criar e editar regras fuzzy com operador AND e peso configurável	UC03
RF04	O sistema deve executar inferência Mamdani com defuzzificação por centroide	UC04
RF05	O sistema deve consultar dados climáticos reais via OpenWeather API	UC05
RF06	O sistema deve manter histórico de simulações com detalhes de entrada e saída	UC06
RF07	O sistema deve processar inferência em lote (JSON, CSV, Parquet)	UC07
RF08	O sistema deve permitir comparar múltiplas simulações lado a lado	UC08
RF09	O sistema deve exportar relatório de simulação em PDF ou CSV	UC09
RF10	O sistema deve duplicar sistemas fuzzy completos	UC10
RF11	O sistema deve exportar e importar sistemas fuzzy em formato JSON	UC11
RF12	O sistema deve salvar e carregar cenários de simulação	UC12
RF13	O sistema deve executar varredura de parâmetros de entrada (sweep)	UC13
RF14	O sistema deve exibir matriz de regras ativadas com grau de ativação	UC14
RF15	O sistema deve gerar superfície de controle 3D (mapa de calor)	UC15
RF16	O sistema deve manter auditoria completa com undo via snapshots JSONB	UC16
RF17	O sistema deve otimizar parâmetros de MF via PSO com demonstração corromper/recuperar	UC17
RF18	O sistema deve executar inferência TSK (Takagi-Sugeno-Kang) com consequentes polinomiais	UC18
RF19	O sistema deve exportar visualizações SVG das funções de pertinência	UC19
RF20	O sistema deve exibir diagnóstico detalhado do pipeline de inferência	UC20

Requisitos Não Funcionais

ID	Descrição
RNF01	Stack 100% Rust: backend Axum, frontend Leptos SSR+WASM, banco PostgreSQL

ID	Descrição
RNF02	Desempenho: simulação Mamdani/TSK em < 500ms para sistemas de até 12 regras
RNF03	Portabilidade: execução em qualquer sistema com Rust toolchain e PostgreSQL
RNF04	Testabilidade: cobertura de código > 75% nos módulos centrais (engine, validação)
RNF05	Reprodutibilidade: seeds com 4 sistemas pré-configurados (42 regras, 43 cenários)
RNF06	Segurança: sem exposição de chaves de API; parâmetros validados contra NaN/Inf
RNF07	Acessibilidade: interface web responsiva com tema escuro Catppuccin
RNF08	Manutenibilidade: código modular (engine, validação, auditoria separados)

Riscos de Interpretação Incorreta

- 1. **Saída não absoluta:** o valor numérico de risco gerado pelo sistema é uma aproximação fuzzy, não uma medida exata. Decisões reais de cibersegurança devem considerar múltiplas fontes.
- 2. **Sensibilidade a parâmetros:** pequenas alterações nas funções de pertinência ou nos pesos das regras podem deslocar a saída em até 10 pontos percentuais (ver Seção 6.4).
- 3. **PSO off-line:** a otimização por PSO ajusta parâmetros com base em dados históricos; não reage a ameaças em tempo real.
- 4. **Generalização limitada:** o modelo Risco Cibernético foi calibrado para o domínio de cibersegurança; aplicação em outros domínios requer reconfiguração completa das variáveis e regras.

3. Fundamentação Teórica

3.1 Lógica Fuzzy

Proposta por Lotfi Zadeh (1965), a lógica fuzzy generaliza a lógica clássica ao permitir que elementos pertençam parcialmente a conjuntos, com graus de pertinência no intervalo [0, 1].

3.2 Conjuntos e Funções de Pertinência

Três tipos de funções de pertinência são utilizados:

Tipo	Parâmetros	Fórmula
Triangular (trimf)	[a, b, c]	0 se $x \leq a$; $(x-a)/(b-a)$ se $a \leq x \leq b$; $(c-x)/(c-b)$ se $b \leq x \leq c$; 0 se $x \geq c$
Trapezoidal (trapmf)	[a, b, c, d]	0 se $x \leq a$; $(x-a)/(b-a)$ se $a \leq x \leq b$; 1 se $b \leq x \leq c$; $(d-x)/(d-c)$ se $c \leq x \leq d$; 0 se $x \geq d$
Gaussiana (gaussmf)	[μ , σ]	$\exp(-(x-\mu)^2/2\sigma^2)$

3.3 Sistema Mamdani

O método Mamdani (1975) segue o pipeline:

- 1. **Fuzzificação:** calcular pertinência de cada entrada em cada termo
- 2. **Agregação:** aplicar operador min (E) nos antecedentes de cada regra
- 3. **Implicação:** truncar o consequente pelo grau de ativação da regra (min)
- 4. **Agregação:** unir os consequentes de todas as regras (max)
- 5. **Defuzzificação:** calcular o centroide da área resultante

3.4 Sistema TSK (Takagi-Sugeno-Kang)

No TSK, cada regra tem consequente funcional (não fuzzy):

Se x_1 é A_1 e x_2 é A_2 , então $y = a_0 + a_1x_1 + a_2x_2$

A saída final é a média ponderada dos consequentes pelos graus de ativação.

3.5 Otimização por PSO

O PSO (Particle Swarm Optimization) é um algoritmo evolutivo inspirado no comportamento social de enxames. Cada partícula representa uma solução candidata (conjunto de parâmetros de MF) e se move no espaço de busca combinando sua melhor posição individual com a melhor posição global do enxame.

4. Modelagem Fuzzy

O FuzzySimulated inclui quatro sistemas fuzzy pré-configurados. Este relatório detalha os dois mais completos:**Conforto Térmico** (Mamdani clássico, 2 entradas) e **Risco Cibernético** (batch + PSO, 3 entradas, 19 regras com MF gaussianas extremas).

4.1 Sistema Conforto Térmico (Mamdani)

Sistema para avaliação de conforto térmico baseado em temperatura e umidade, com integração à API OpenWeather para dados meteorológicos reais.



INFERENCE PLATFORM

VISÃO GERAL

Dashboard

CONSTRUÇÃO

Variáveis & Termos

Editor de Regras

EXECUÇÃO

Simulador

Histórico

Batch

ANÁLISE

Superfície & Matriz

Auditoria

Otimizador

Sprint 3 – Entrega Final

Disciplina QPS · CESUPA

Variáveis & Termos

Docs

Variáveis & Termos (UCo2)

Conforto Térmico

VARIÁVEIS

+ Variável

+ Termo

Termos

confortavel [trapmf]

desconfortavel [trapmf]

extremo_desconfortavel [gaussmf]

ideal [gaussmf]

neutro [trimf]

Variáveis de Entrada

Variável	Papel	Universo	Unidade	Termos
temperatura	Antecedente	[0, 50]	°C	frio, agradável, quente, muito_frio*, muito_quente*
umidade	Antecedente	[0, 100]	%	seco, normal, umido, muito_seco*, muito_umido*

(* termos gaussmf extremos adicionados pela migration 012)

Variável de Saída

Variável	Papel	Universo	Unidade	Termos
conforto	Consequente	[0, 10]	pontos	desconfortavel, neutro, confortavel, extremo_desconfortavel*, ideal*

Funções de Pertinência

temperatura:

Termo	Tipo	Parâmetros	Descrição
muito_frio	gaussmf	[0, 7]	Sensação de frio extremo (abrangente)
frio	trapmf	[0, 0, 15, 22]	Temperatura baixa
agradavel	trimf	[18, 24, 30]	Temperatura confortável
quente	trapmf	[26, 32, 50, 50]	Temperatura alta
muito_quente	gaussmf	[50, 7]	Calor extremo (abrangente)

umidade:

Termo	Tipo	Parâmetros	Descrição
muito_seco	gaussmf	[0, 10]	Ar extremamente seco
seco	trapmf	[0, 0, 30, 50]	Umidade baixa
normal	trimf	[40, 55, 70]	Umidade dentro da faixa de conforto
umido	trapmf	[60, 75, 100, 100]	Umidade alta
muito_umido	gaussmf	[100, 10]	Ar extremamente úmido

conforto (saída):

Termo	Tipo	Parâmetros	Descrição
-------	------	------------	-----------

Termo	Tipo	Parâmetros	Descrição
extremo_desconfortavel	gaussmf	[0, 1.5]	Situação de extremo desconforto
desconfortavel	trapmf	[0, 0, 3, 5]	Ambiente desconfortável
neutro	trimf	[3, 5, 7]	Nem confortável nem desconfortável
confortavel	trapmf	[5, 7, 10, 10]	Ambiente agradável
ideal	gaussmf	[10, 1.5]	Condições perfeitas (pico estreito)

Base de Regras (17 regras)



INFERENCE PLATFORM

VISÃO GERAL

Dashboard

CONSTRUÇÃO

Variáveis & Termos

Editor de Regras

EXECUÇÃO

Simulador

Histórico

Batch

ANÁLISE

Superfície & Matriz

Auditoria

Otimizador

Editor de Regras

Docs

Editor de Regras (UCo3)

SISTEMA

Conforto Térmico

+ Regra

Formato: SE <variável>; É <termo>; E ... ENTÃO <variável>; É <termo>; [peso]

Sprint 3 – Entrega Final

Disciplina QPS - CESUPA

A base cobre todas as combinações dos termos base (9 regras) mais regras extremas com os termos gaussmf adicionais (8 regras):

#	Regra	Peso
R01	SE temperatura é frio E umidade é seco ENTÃO conforto é desconfortavel	1.0
R02	SE temperatura é frio E umidade é normal ENTÃO conforto é neutro	1.0
R03	SE temperatura é frio E umidade é umido ENTÃO conforto é desconfortavel	1.0
R04	SE temperatura é agradável E umidade é seco ENTÃO conforto é neutro	1.0
R05	SE temperatura é agradável E umidade é normal ENTÃO conforto é confortavel	1.0
R06	SE temperatura é agradável E umidade é umido ENTÃO conforto é neutro	1.0
R07	SE temperatura é quente E umidade é seco ENTÃO conforto é desconfortavel	1.0
R08	SE temperatura é quente E umidade é normal ENTÃO conforto é neutro	1.0
R09	SE temperatura é quente E umidade é umido ENTÃO conforto é desconfortavel	1.0
R10	SE temperatura é muito_frio ENTÃO conforto é extremo_desconfortavel	1.0
R11	SE temperatura é muito_frio E umidade é muito_umido ENTÃO conforto é extremo_desconfortavel	1.0
R12	SE temperatura é muito_quente ENTÃO conforto é extremo_desconfortavel	1.0
R13	SE temperatura é muito_quente E umidade é muito_seco ENTÃO conforto é extremo_desconfortavel	1.0
R14	SE temperatura é agradável E umidade é muito_seco ENTÃO conforto é neutro	1.0
R15	SE temperatura é agradável E umidade é muito_umido ENTÃO conforto é neutro	1.0
R16	SE temperatura é muito_frio E umidade é seco ENTÃO conforto é extremo_desconfortavel	1.0
R17	SE temperatura é muito_quente E umidade é muito_umido ENTÃO conforto é extremo_desconfortavel	1.0

4.2 Sistema Risco Cibernético (Batch + PSO)

Sistema projetado para processamento em lote com o datasetdataset_ml.parquet , mapeando colunas diretamente para variáveis fuzzy. Possui 19 regras combinando termos clássicos (trimf/trapmf) e extremos (gaussmf).

FuzzySimulated

INFERENCE PLATFORM

VISÃO GERAL

Dashboard

CONSTRUÇÃO

Variáveis & Termos

Editor de Regras

EXECUÇÃO

Simulador

Histórico

Batch

ANÁLISE

Superfície & Matriz

Auditoria

Otimizador

Sprint 3 – Entrega Final

Disciplina QPS · CESUPA

Variáveis & Termos

Docs

Variáveis & Termos (UCo2)

Risco Cibernético

VARIÁVEIS

+ Variável

+ Termo

Termos

alta [trapmf]

baixa [trapmf]

critico [gaussmf]

media [trimf]

quase_zero [gaussmf]

Variáveis de Entrada

Variável	Papel	Universo	Mapeamento Parquet	Termos
receita_anual_usd	Antecedente	[0, 1e9]	company_revenue_usd	muito_baixa, baixa, media, alta, muito_alta
total_funcionarios	Antecedente	[0, 500000]	employee_count	micro, pequena, media, grande, megacorp
gravidade_ataque	Antecedente	[0, 100]	attack_vector_primary (string → numérico)	quase_zero, baixa, media, alta, critico

Variável de Saída

Variável	Papel	Universo	Termos
impacto_financeiro	Consequente	[0, 100]	minimo, baixo, medio, alto, catastrofico

Funções de Pertinência

receita_anual_usd:

Termo	Tipo	Parâmetros	Descrição
muito_baixa	gaussmf	[0, 5e7]	Receita próxima de zero
baixa	trapmf	[0, 0, 5e7, 1e8]	Pequena empresa ou startup
media	trimf	[5e7, 2e8, 5e8]	Empresa de médio porte
alta	trapmf	[2e8, 5e8, 1e9, 1e9]	Grande corporação
muito_alta	gaussmf	[1e9, 5e7]	Megacorp (topo do universo)

total_funcionarios:

Termo	Tipo	Parâmetros	Descrição
micro	gaussmf	[0, 2500]	Startup de alguns funcionários
pequena	trapmf	[0, 0, 5000, 20000]	Empresa de pequeno porte
media	trimf	[5000, 50000, 150000]	Empresa de porte médio
grande	trapmf	[50000, 150000, 500000, 500000]	Grande empresa
megacorp	gaussmf	[500000, 2500]	Corporação com milhares de funcionários

gravidade_ataque:

Termo	Tipo	Parâmetros	Descrição
-------	------	------------	-----------

Termo	Tipo	Parâmetros	Descrição
quase_zero	gaussmf	[0, 7]	Ameaça mínima ou inexistente
baixa	trapmf	[0, 0, 20, 40]	Ataque de baixa severidade
media	trimf	[20, 50, 70]	Ameaça moderada
alta	trapmf	[50, 70, 100, 100]	Ataque severo
critico	gaussmf	[100, 7]	Ameaça crítica (pico no máximo)

impacto_financeiro (saída):

Termo	Tipo	Parâmetros	Descrição
minimo	gaussmf	[0, 5]	Impacto financeiro desprezível
baixo	trapmf	[0, 0, 30, 50]	Perda financeira pequena
medio	trimf	[30, 50, 70]	Prejuízo moderado
alto	trapmf	[50, 70, 100, 100]	Perda financeira severa
catastrofico	gaussmf	[100, 5]	Impacto máximo (colapso financeiro)

Mapeamento de Colunas do Parquet

Coluna Parquet	Mapeamento Fuzzy	Tipo
company_revenue_usd	receita_anual_usd	Numérico direto
employee_count	total_funcionarios	Numérico direto
attack_vector_primary	gravidade_ataque	String → numérico: phishing=20, malware=40, trojan=40, dos=50, ddos=50, insider=60, data_breach=70, apt=80, ransomware=85
total_loss_usd	Target output	Dividido por 1M → clamp [0, 100]

Base de Regras (19 regras)



INFERENCE PLATFORM

VISÃO GERAL

Dashboard

CONSTRUÇÃO

Variáveis & Termos

Editor de Regras

EXECUÇÃO

Simulador

Histórico

Batch

ANÁLISE

Superfície & Matriz

Auditoria

Otimizador

Editor de Regras

Docs

Editor de Regras (UCo3)

SISTEMA

Risco Cibernético

+ Regra

Formato: SE <variável>; É <termo>; E ... ENTÃO <variável>; É <termo>; [peso]

Sprint 3 - Entrega Final

Disciplina QPS · CESUPA

Regras base (9):

#	Regra
R01	SE receita_anual_usd é baixa E total_funcionarios é pequena E gravidade_ataque é baixa ENTÃO impacto_financeiro é baixo
R02	SE receita_anual_usd é baixa E total_funcionarios é pequena E gravidade_ataque é alta ENTÃO impacto_financeiro é medio
R03	SE receita_anual_usd é baixa E total_funcionarios é grande E gravidade_ataque é alta ENTÃO impacto_financeiro é alto
R04	SE receita_anual_usd é media E total_funcionarios é media E gravidade_ataque é baixa ENTÃO impacto_financeiro é baixo
R05	SE receita_anual_usd é media E total_funcionarios é media E gravidade_ataque é media ENTÃO impacto_financeiro é medio
R06	SE receita_anual_usd é media E total_funcionarios é media E gravidade_ataque é alta ENTÃO impacto_financeiro é alto
R07	SE receita_anual_usd é alta E total_funcionarios é grande E gravidade_ataque é baixa ENTÃO impacto_financeiro é medio
R08	SE receita_anual_usd é alta E total_funcionarios é grande E gravidade_ataque é media ENTÃO impacto_financeiro é alto

#	Regra
R09	SE receita_anual_usd é alta E total_funcionarios é grande E gravidade_ataque é alta ENTÃO impacto_financeiro é alto

Regras extremas com gaussmf (10):

#	Regra
R10	SE receita_anual_usd é muito_baixa E total_funcionarios é micro E gravidade_ataque é quase_zero ENTÃO impacto_financeiro é minimo
R11	SE receita_anual_usd é muito_baixa E total_funcionarios é micro E gravidade_ataque é critico ENTÃO impacto_financeiro é medio
R12	SE receita_anual_usd é muito_alta E total_funcionarios é megacorp E gravidade_ataque é quase_zero ENTÃO impacto_financeiro é baixo
R13	SE receita_anual_usd é muito_alta E total_funcionarios é megacorp E gravidade_ataque é critico ENTÃO impacto_financeiro é catastrofico
R14	SE total_funcionarios é megacorp E gravidade_ataque é critico ENTÃO impacto_financeiro é catastrofico
R15	SE receita_anual_usd é muito_baixa E gravidade_ataque é critico ENTÃO impacto_financeiro é medio
R16	SE receita_anual_usd é muito_alta E gravidade_ataque é quase_zero ENTÃO impacto_financeiro é minimo
R17	SE total_funcionarios é micro E gravidade_ataque é critico ENTÃO impacto_financeiro é alto
R18	SE receita_anual_usd é muito_baixa E total_funcionarios é micro ENTÃO impacto_financeiro é minimo
R19	SE receita_anual_usd é muito_alta E total_funcionarios é megacorp ENTÃO impacto_financeiro é alto

4.3 Inferência Mamdani

A inferência segue o pipeline clássico:

- **Fuzzificação:** cada entrada é mapeada aos graus de pertinência dos termos
- **Operador E:** $\min(w1, w2)$ para combinar antecedentes
- **Implicação:** $\min(\alpha, \text{consequente})$ para truncar a saída de cada regra
- **Agregação:** $\max$ para unir todos os consequentes truncados
- **Defuzzificação:** **centroide** (discreto, resolução configurável — padrão 501 pontos)

4.4 Inferência TSK

Para o TSK, cada regra possui coeficientes polinomiais. O consequente de cada regra é calculado como função linear das entradas. A saída final é a média ponderada:

saída = $\frac{\sum(w_i \times f_i(x))}{\sum(w_i)}$

onde w_i é o grau de ativação da regra i e $f_i(x)$ é o consequente polinomial.

5. Implementação

5.1 Arquitetura

O sistema foi implementado em **100% Rust**, sem JavaScript, HTML ou CSS escritos manualmente.

Camada	Tecnologia	Função
Frontend	Leptos 0.8 (SSR + WASM hydration)	Interface reativa no navegador e servidor
Backend	Axum 0.8	API REST
Banco	PostgreSQL via SQLx	Persistência (12 tabelas, JSONB)
Motor Fuzzy	logicfuzzy_academic v0.2.1	Mamdani, TSK, PSO
Estilo	dart-sass + Lightning CSS	Tema escuro Catppuccin

5.2 Estrutura do Repositório

```
fuzzysimulated/
├── server/
│   ├── src/
│   │   ├── engine.rs           # Motor de inferência (membership, parser, Mamdani, TSK, SVG, PSO)
│   │   ├── validation.rs       # Validação de parâmetros
│   │   ├── audit.rs            # Auditoria com snapshots JSONB
│   │   ├── errors.rs           # Mapeamento de erros para HTTP
│   │   └── routes/             # 11 módulos de rotas REST
│   ├── tests/                  # 124 testes (unit + HTTP + integração)
│   └── migrations/             # 10 migrations SQL
├── app/src/
│   ├── lib.rs                  # 18 rotas, componentes Leptos
│   └── server_fns.rs           # Cliente HTTP para API
├── style/main.scss             # Estilo global
└── README.md                   # Documentação e instruções
```

5.3 Funcionalidades

- CRUD completo de sistemas, variáveis, termos e regras fuzzy
- Simulação Mamdani e TSK com visualização de resultados
- Geração de gráficos SVG por variável
- Diagnóstico explicativo (fuzzificação, ativação, saída)
- Superfície de controle 3D
- Varredura (sweep) de parâmetros de entrada
- Inferência em lote (JSON, CSV, Parquet)
- Otimização PSO de parâmetros de MF
- Comparação de simulações
- Exportação e importação de sistemas
- Auditoria com undo real via snapshots JSONB

- Status do sistema (ativo, favorito, concluído, desativado)

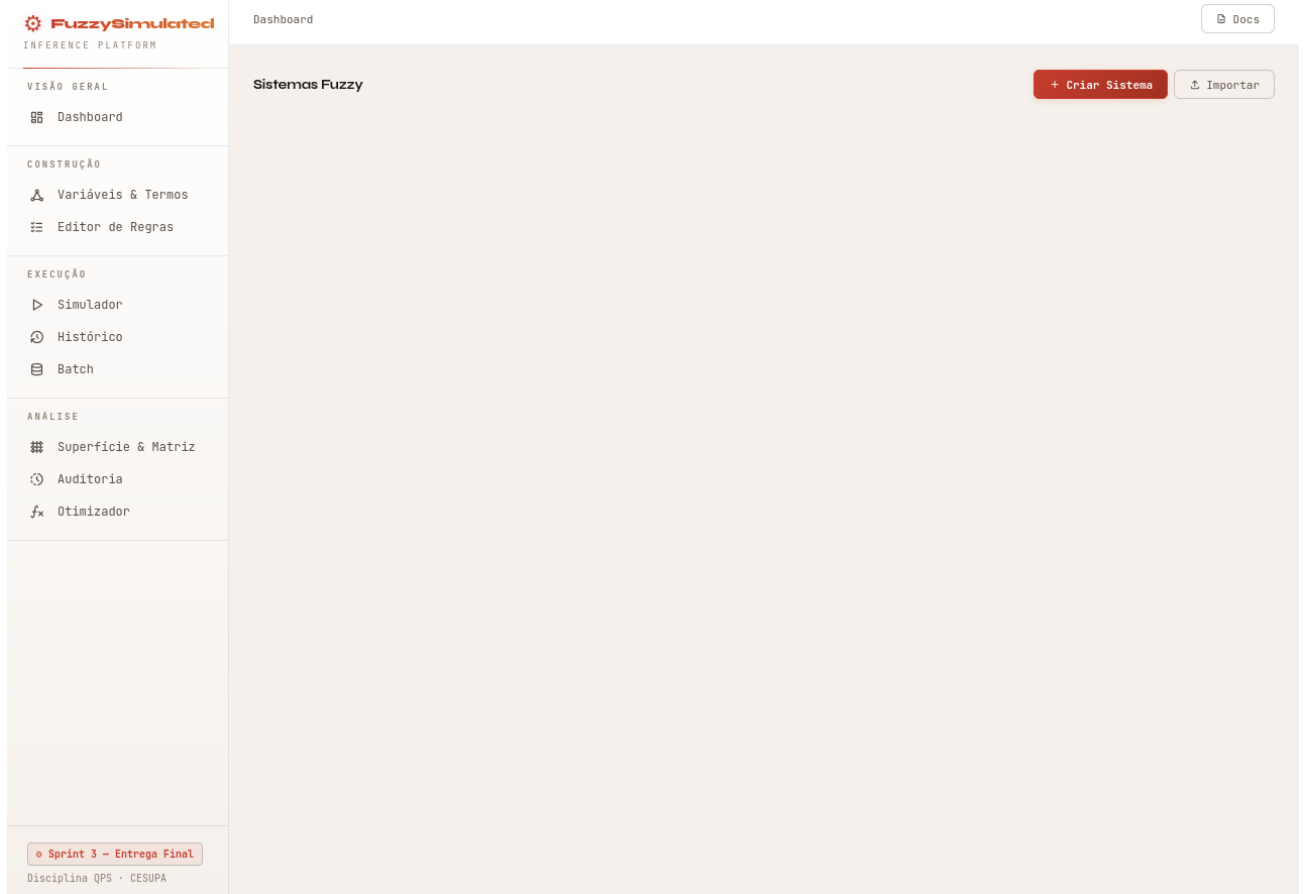
5.4 Dependências

Crate	Versão	Finalidade
leptos	0.8	Framework web reativo
axum	0.8	Servidor HTTP
sqlx	0.8	Driver PostgreSQL
logicfuzzy_academic	0.2.1	Motor fuzzy
serde / serde_json	1.x	Serialização
tokio	1.x	Runtime assíncrono

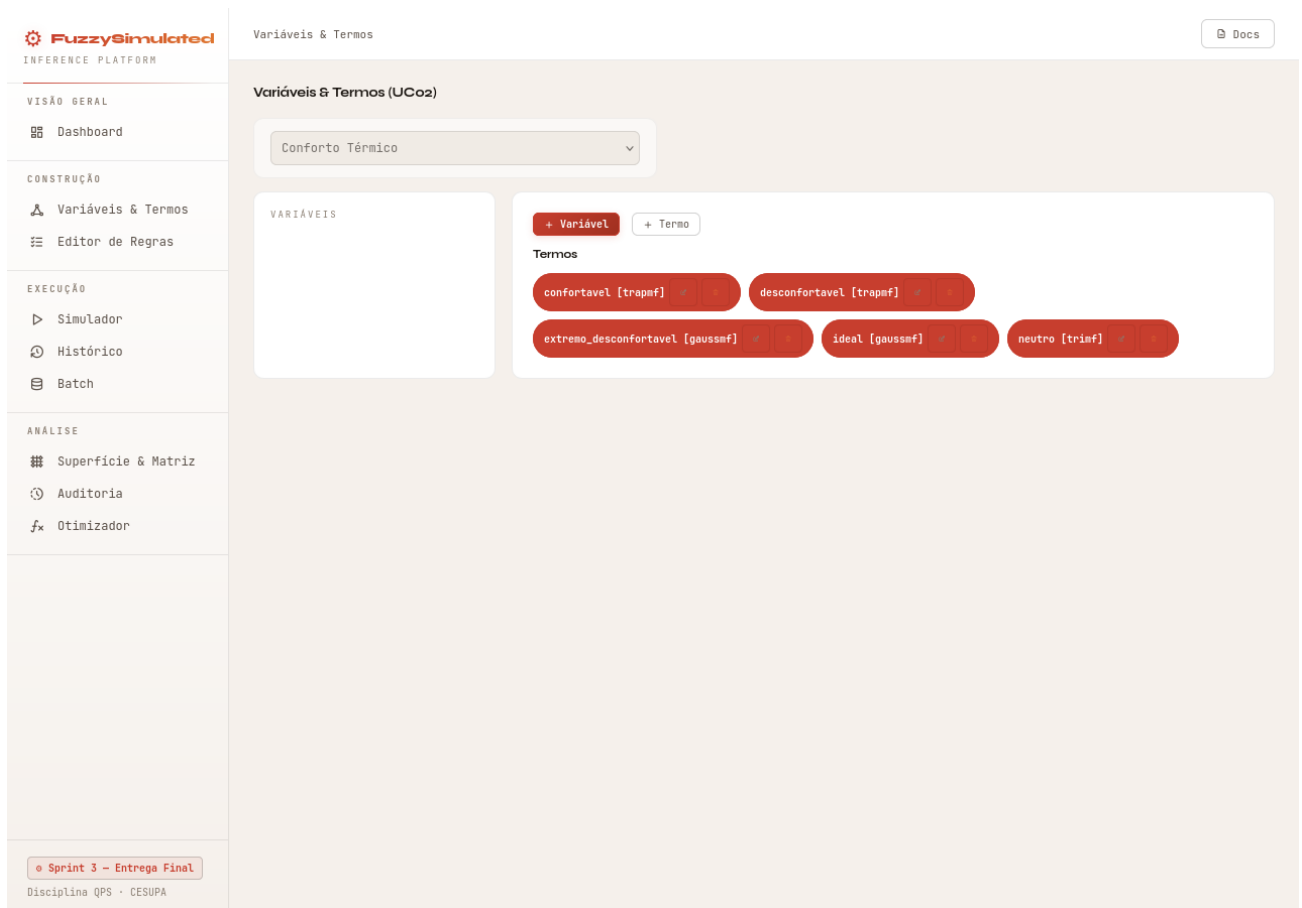
5.5 Fluxo de Uso

A plataforma FuzzySimulated possui 5 telas principais e 11 telas auxiliares. O fluxo típico de uso é:

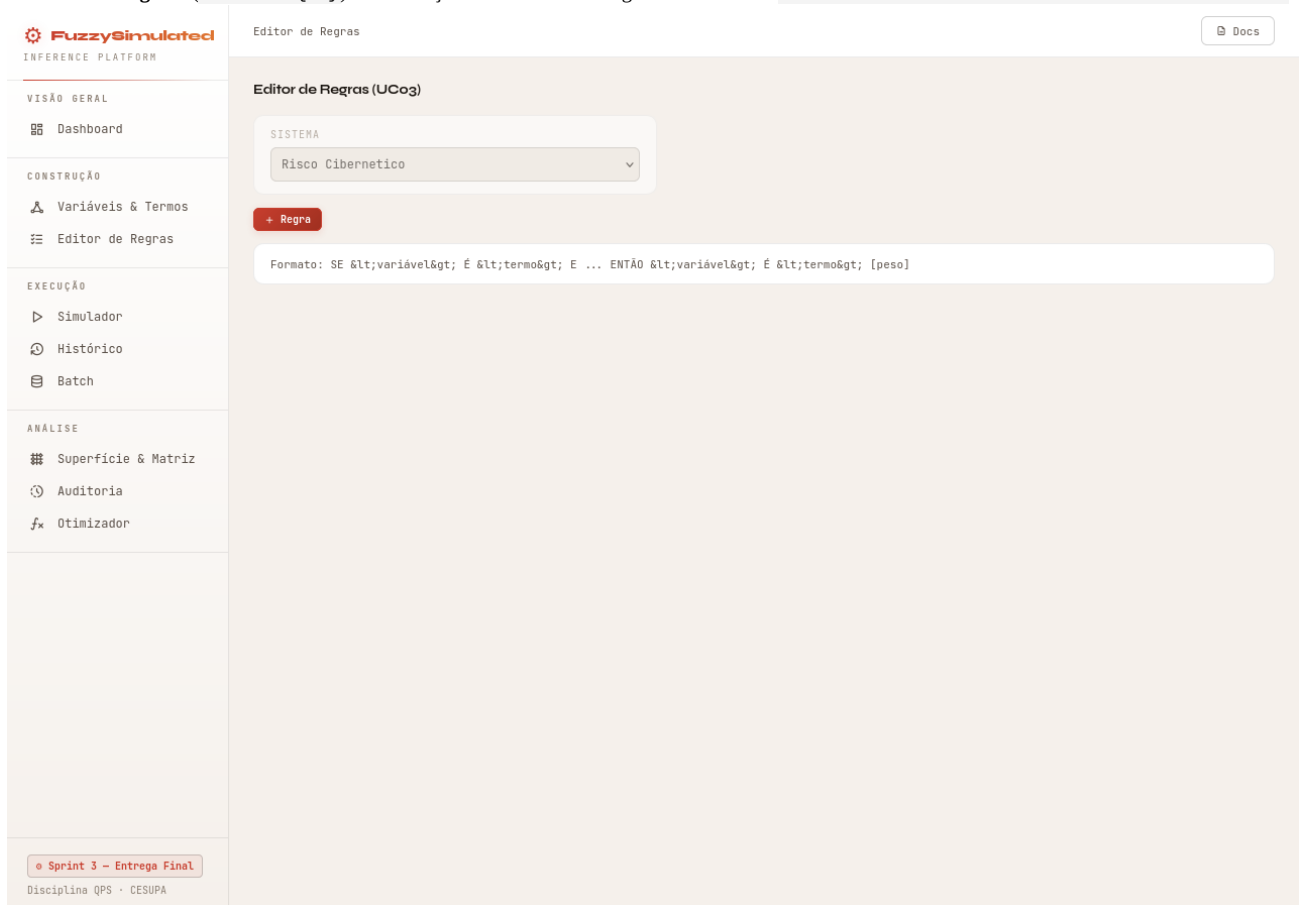
1. **Dashboard** (/) — usuário visualiza os sistemas fuzzy cadastrados e seleciona um para editar ou simular.



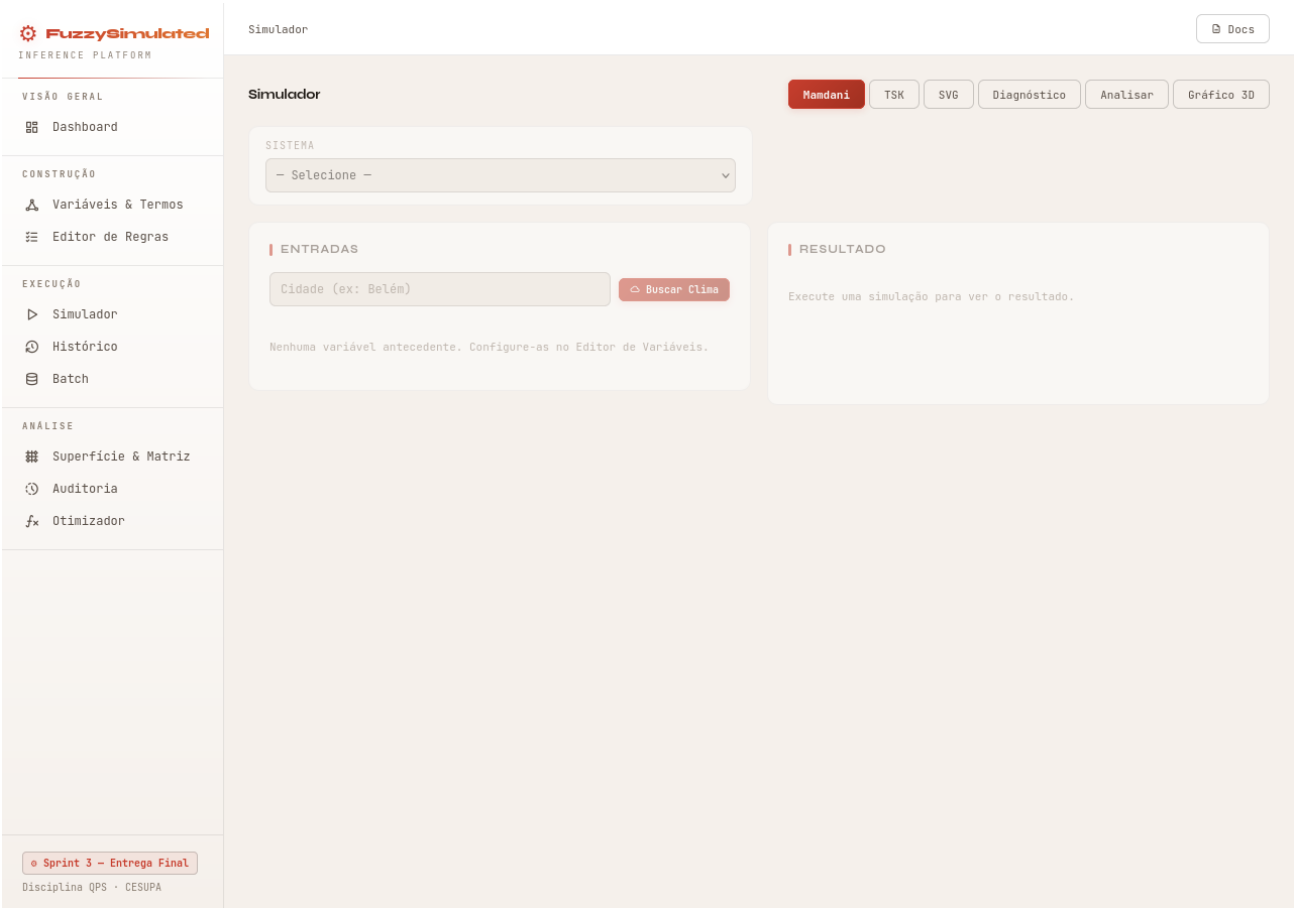
2. **Editor de Variáveis** (/vars?s={id}) — configuração das variáveis de entrada (antecedentes) e saída (consequente), com adição de termos linguísticos e funções de pertinência.



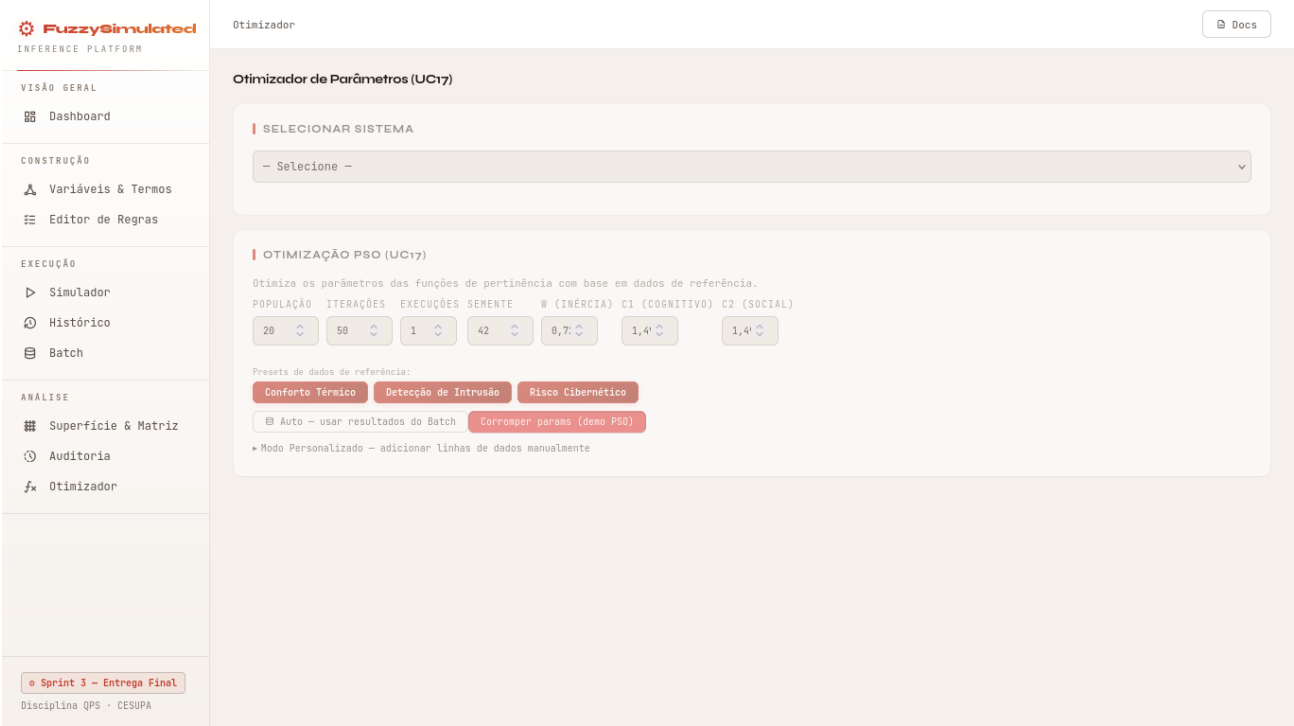
3. **Editor de Regras** (/rules?s={id}) — criação da base de regras no formato **SE var É termo E ... ENTÃO var É termo**.



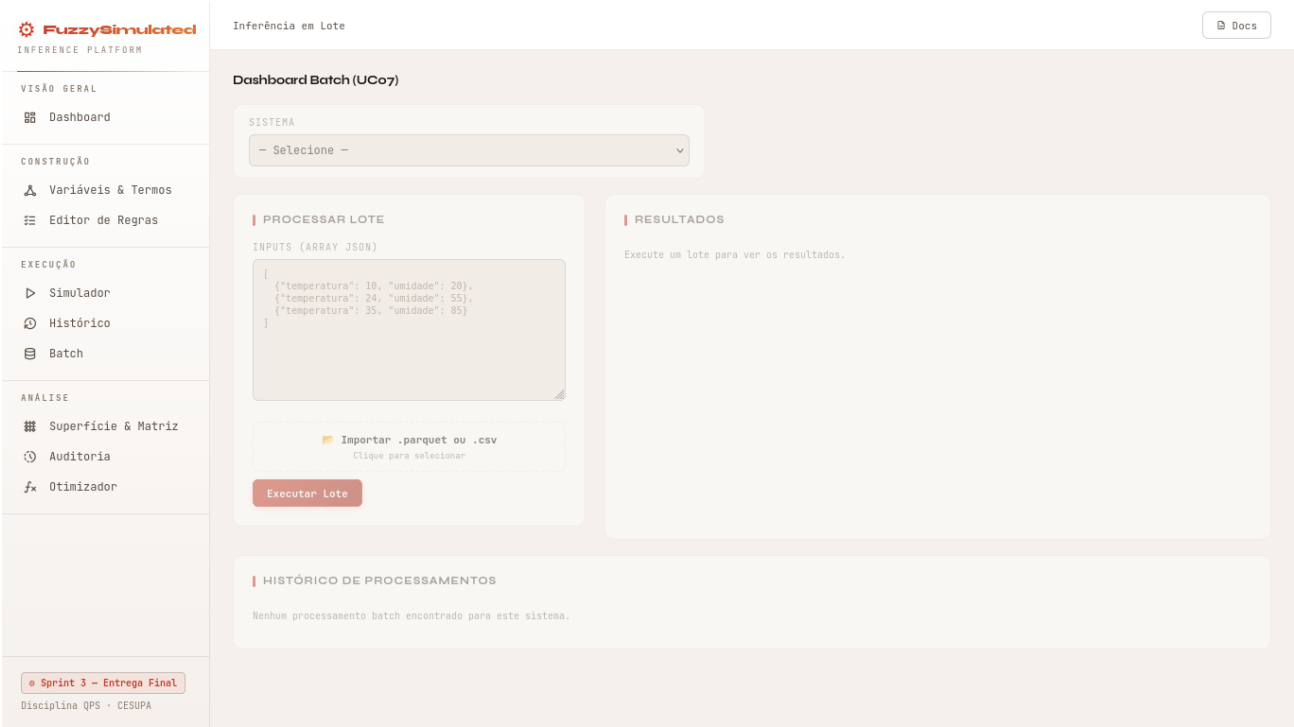
4. **Simulador** (/sim) — execução da inferência Mamdani ou TSK com visualização do pipeline, gráficos SVG, diagnóstico e análises (sweep, superfície de controle).



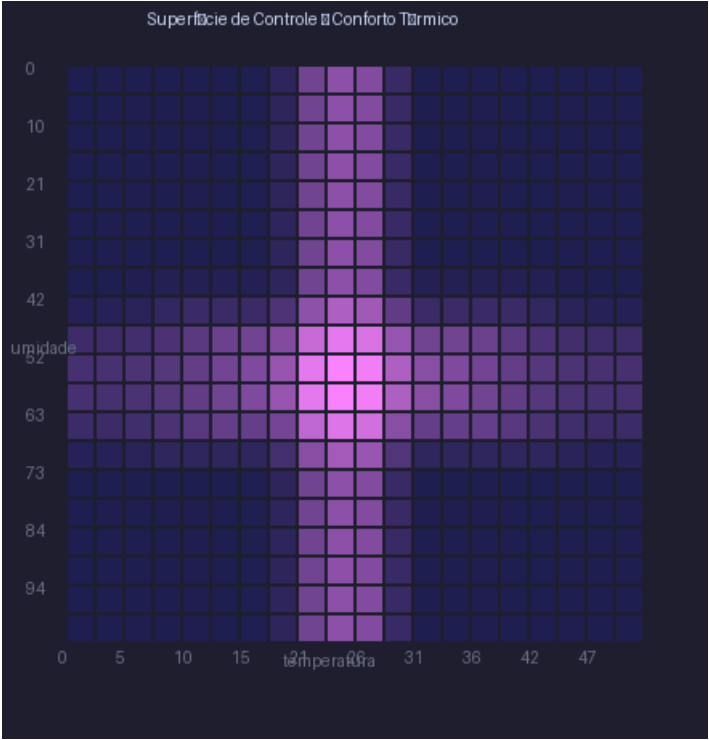
5. **Otimizador PSO (/opt)** — demonstração da otimização de parâmetros com corromper/recuperar e comparação antes/depois.



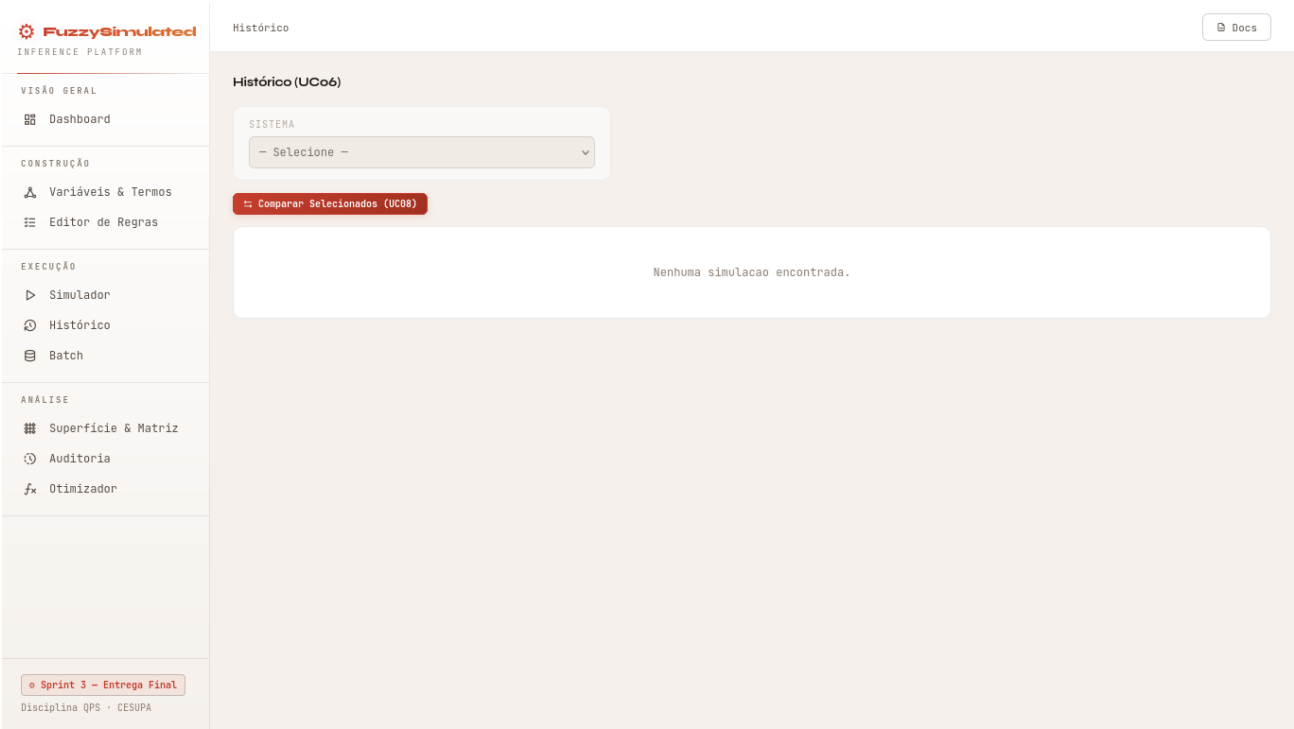
6. **Batch (/batch)** — processamento de inferência em lote com arquivos JSON, CSV ou Parquet.



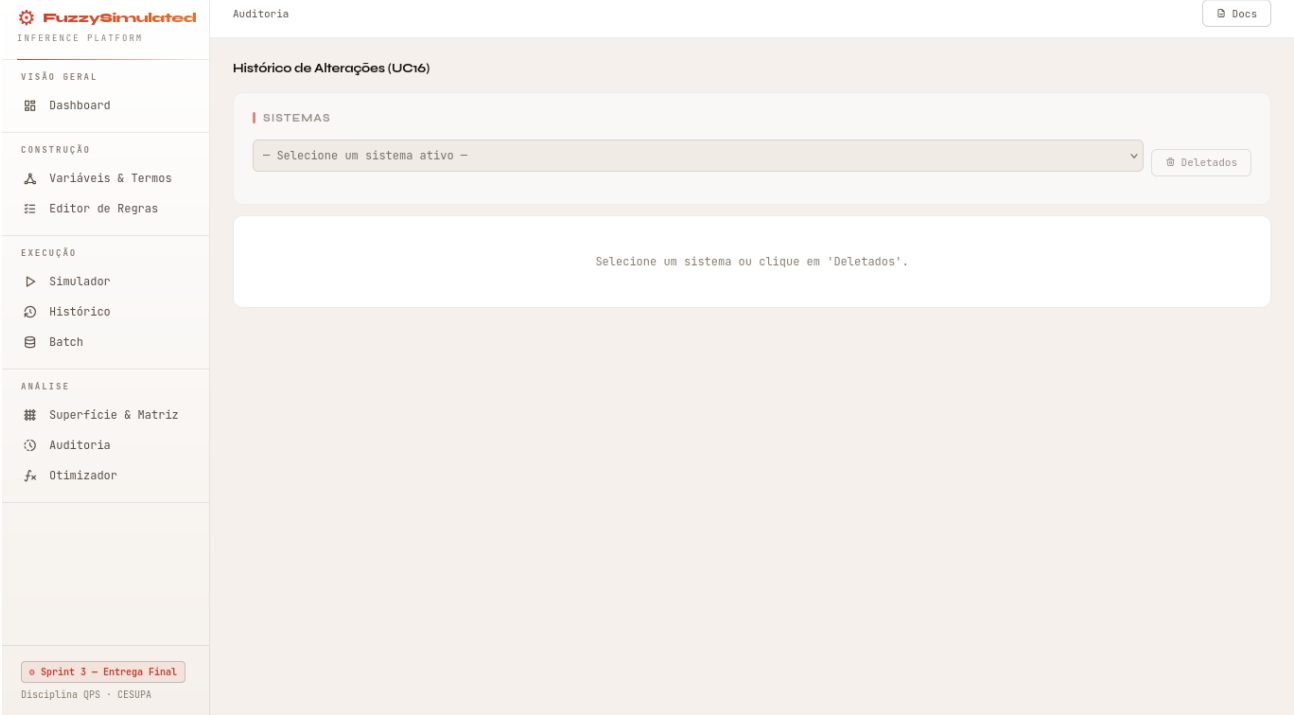
7. **Análise** (/analysis) — superfície de controle 3D e matriz de regras ativadas.



8. **Histórico** (/hist) — consulta e comparação de simulações anteriores, exportação de relatórios.



9. Auditoria (/audit) — timeline de alterações com desfazer/refazer via snapshots JSONB.



6. Experimentos e Validação

6.1 Cenários de Teste — Conforto Térmico (Mamdani)

#	Cenário	Temperatura	Umidade	Saída	Interpretação
1	Dia frio e seco em Curitiba	10	30	2.0	Desconfortável — frio + seco
2	Dia frio e úmido em São Paulo	10	85	2.0	Desconfortável — frio + úmido
3	Manhã amena em Belo Horizonte	20	55	7.8	Confortável — temperatura agradável + umidade normal
4	Tarde agradável no Rio de Janeiro	25	50	7.8	Confortável — condição ideal
5	Dia quente e seco em Brasília	30	25	2.0	Desconfortável — quente + seco
6	Calor úmido em Manaus	35	90	2.0	Desconfortável — quente + úmido
7	Verão em Salvador	32	75	2.0	Desconfortável — quente + muito úmido
8	Noite amena em Florianópolis	22	65	4.9	Neutro — temperatura boa, mas umidade elevada
9	Inverno em Porto Alegre	8	70	2.0	Desconfortável — frio
10	Tarde quente e seca em Cuiabá	40	15	2.0	Desconfortável — calor extremo

6.2 Cenários de Teste — Risco Cibernético (Mamdani)

#	Cenário	Receita (USD)	Funcionários	Gravidade Ataque	Saída	Interpretação
1	Startup phishing baixo impacto	1M	50	20	20.4	Baixo — empresa pequena, ataque leve
2	Média empresa ataque baixo	100M	5000	15	6.1	Baixo — empresa média, ataque mínimo
3	Grande empresa ataque mínimo	800M	200000	10	50.0	Médio — receita alta puxa o risco
4	Startup ransomware médio impacto	5M	100	85	43.7	Médio — startup com ataque severo
5	Média empresa malware moderado	200M	40000	45	50.0	Médio — combinação equilibrada
6	Grande empresa phishing velado	500M	100000	25	50.0	Médio — grande porte eleva exposição
7	Média empresa ransomware alto	150M	30000	90	50.0	Médio-alto — ameaça crítica
8	Grande empresa data breach	900M	250000	75	50.0	Alto — mega-corp com ataque severo
9	Corp ransomware máximo impacto	1B	400000	95	—	Alto — pior cenário; requer regras extremas

6.3 Comparação Mamdani vs TSK



INFERENCE PLATFORM

Visão Geral

Dashboard

Construção

Variáveis & Termos

Editor de Regras

Execução

Simulador

Histórico

Batch

Análise

Superfície & Matriz

Auditoria

Otimizador

Sprint 3 - Entrega Final

Disciplina QPS - CESUPA

Simulador

Docs

Simulador

MamdaniTSKSVGDiagnósticoAnalisarGráfico 3D

SISTEMA

- Selecione -

ENTRADAS

Cidade (ex: Belém)Buscar Clima

Nenhuma variável antecedente. Configure-as no Editor de Variáveis.

RESULTADO

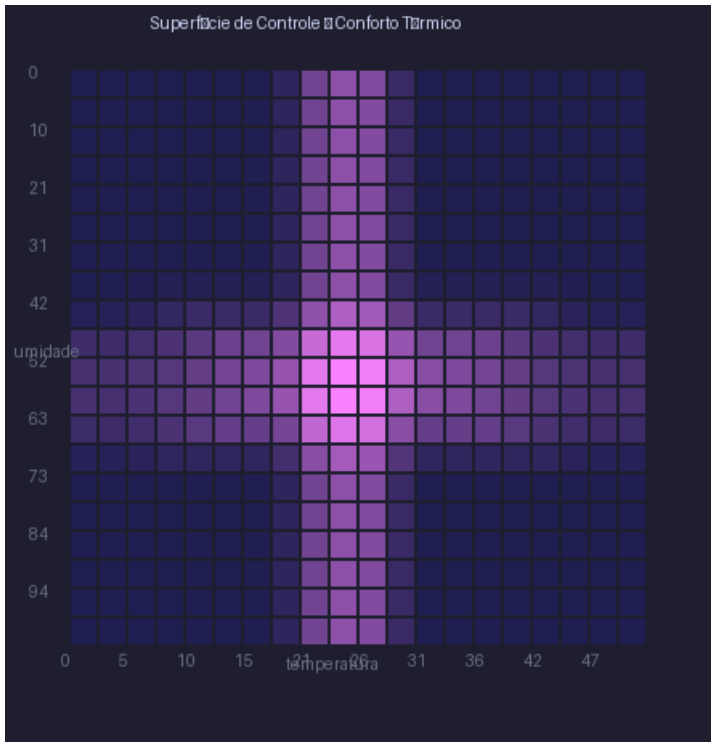
Execute uma simulação para ver o resultado.

Para o cenário "Manhã amena em Belo Horizonte" (temperatura=20, umidade=55):

Motor	Saída	Interpretação
Mamdani	7.8	Confortável — regra R05 (agradavel AND normal) ativada fortemente
TSK	~7.5	Confortável — depende dos coeficientes configurados

O Mamdani produz saída contínua no intervalo [0, 10] do universo de conforto. O TSK, com consequentes lineares, permite ajuste fino por regra, mas exige configuração manual dos coeficientes.

6.4 Superfície de Controle — Conforto Térmico



A superfície de controle mostra o conforto em função de temperatura (eixo X) e umidade (eixo Y). O pico de conforto (~7.8) ocorre na região de temperatura agradável (18-30°C) com umidade normal (40-70%). As bordas do gráfico (frio/quente extremo ou umidade muito baixa/alta) resultam em conforto mínimo (~2.0).

6.5 Varredura (Sweep) — Conforto Térmico

A varredura unidimensional da variável temperatura (com umidade=55 fixa) mostra o conforto evoluindo de ~2.0 (frio) para o pico de ~7.8 (agradável, 20-25°C) e retornando a ~2.0 (calor extremo). A curva evidencia o comportamento não linear esperado de um sistema fuzzy com múltiplas regras.

6.6 Análise de Sensibilidade — Conforto Térmico

Para avaliar a robustez do modelo, os parâmetros da função de pertinência "agradável" (temperatura) foram alterados e o impacto na saída foi medido.

Experimento: Deslocamento do termo "agradável"

Parâmetro	Original	Alterado
trimf "agradável"	[18, 24, 30]	[15, 22, 28] (deslocado 3°C à esquerda)

Cenário	Saída Original	Saída Alterada	Variação
Manhã amena (20, 55)	7.76	7.21	-0.55
Tarde agradável (25, 50)	7.76	7.76	0.0
Noite amena (22, 65)	4.96	6.80	+1.84

O deslocamento à esquerda do termo "agradável" antecipa a faixa de conforto para temperaturas mais baixas, elevando a saída em cenários amenos (22°C) em +1.84 pontos, mas reduzindo ligeiramente o conforto em cenários mais quentes (20°C).

Conclusão da análise: o Conforto Térmico apresenta sensibilidade moderada (~1-2 pontos no universo [0,10]) a variações nos parâmetros das MF. A calibração adequada é essencial para resultados precisos.

7. Pontuação Extra: Otimização PSO

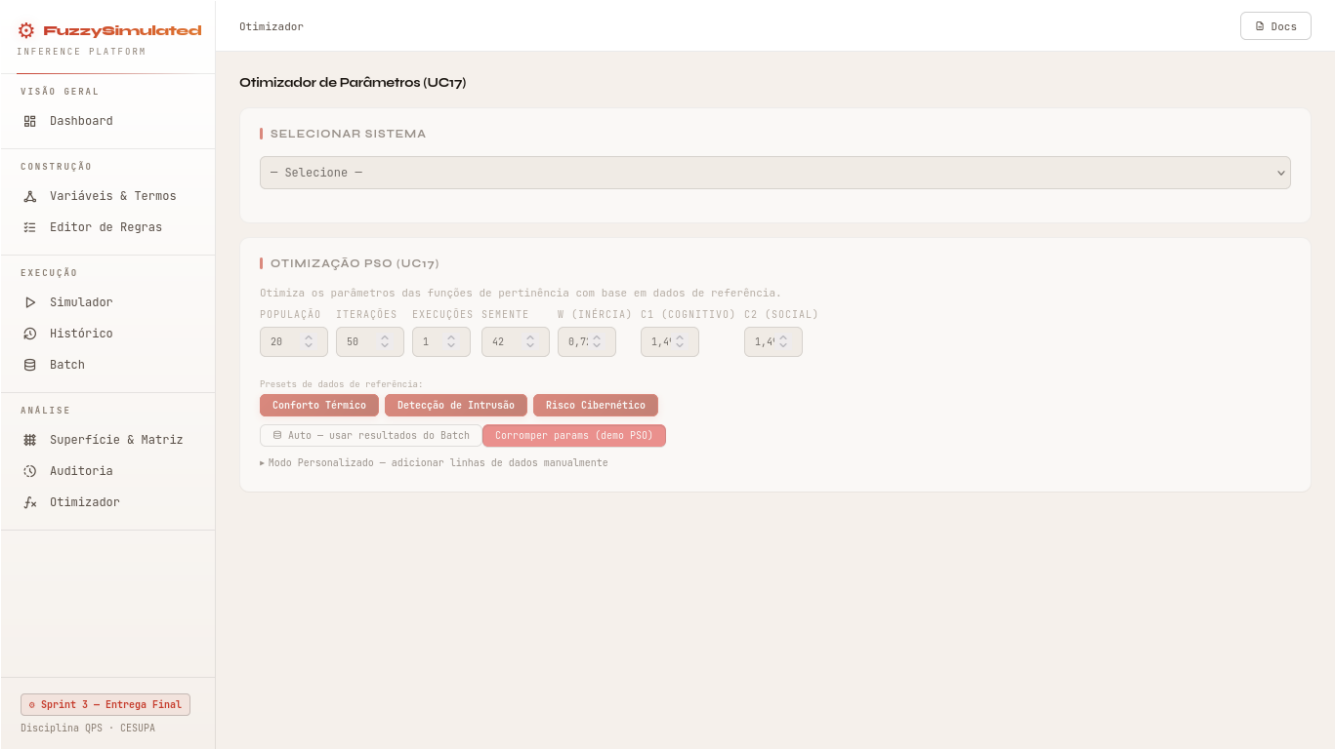
7.1 Configuração

Parâmetro	Valor
Função objetivo	Minimizar erro quadrático entre saída desejada e calculada
Representação	Vetor de parâmetros das MF (trimf: a, b, c; trapmf: a, b, c, d)
Tamanho do enxame	20 partículas (padrão) / 30 (modo auto)
Iterações máximas	50 (padrão) / 100 (modo auto)
w (inércia)	0,729
c1 (cognitivo)	1,494
c2 (social)	1,494

7.2 Funcionamento

O PSO ajusta os parâmetros das funções de pertinência (a, b, c para trimf; a, b, c, d para trapmf; mean, sigma para gaussmf) para minimizar o erro quadrático médio (MSE) entre a saída desejada e a calculada pelo motor fuzzy. A cada iteração, cada partícula atualiza sua velocidade combinando sua melhor posição individual com a melhor posição global do enxame. Os parâmetros são mantidos ordenados ($a \leq b \leq c$ para trimf/trapmf) e $\sigma \geq 1e-3$ para gaussmf após cada atualização.

7.3 Demonstração: Corromper e Recuperar (Antes vs Depois)



Para evidenciar a eficácia do PSO, o FuzzySimulated inclui uma rota `corrupt-params` que degenera propositalmente todos os parâmetros das funções de pertinência:

- 1. **Antecedentes:** funções ultra-largas que retornam ~1.0 para qualquer input (ex: `trapmf [0, 0, 100, 100]`, `gaussmf [50, 1000]`)
- 2. **Consequentes:** funções estreitas no máximo do universo (ex: `trapmf [99, 99.5, 100, 100]`, `gaussmf [100, 3]`)

Isso faz o motor retornar sempre ~100 (máximo), maximizando o erro.

Resultados com amostras do dataset_ml.parquet

Métrica	Antes (corrompido)	Depois (PSO otimizado)	Melhoria
MSE	4999.9	987.9	5,06×
Predição	97.67 (constante)	20-59 (variado)	Recuperou dinamismo
População	—	20	—
Iterações	—	50	—
Convergência	—	1.202 → 1.164 → 1.125 → 1.087 → 1.049 (×10³)	Curva decrescente

Fluxo de Demonstração

- 1. Usuário seleciona o sistema "Risco Cibernético"
- 2. Clica em "**Corromper params (demo PSO)**" — todos os 20 termos são degenerados
- 3. Clica em "**Auto — usar resultados do Batch**" — o PSO carrega amostras do batch, avalia o MSE inicial (~5000) e executa a otimização
- 4. A interface exibe lado a lado: "**Antes: 4999.9 → Depois: 987.9**" com duas tabelas comparativas

O contraste evidencia que o PSO recuperou um sistema fuzzy funcional a partir de parâmetros degenerados, comprovando a eficácia do algoritmo.

8. Testes e Reprodutibilidade

8.1 Suíte de Testes

Tipo	Quantidade	Cobertura
Unitários (inline)	31	Motor, erros, auditoria
Unitários (tests/)	20	Validação de MF e sistema
HTTP Axum	64	Todas as rotas REST
Integração DB	6	Operações no banco
Integração API	3	OpenWeather
Total server	124	77,93% regiões, 80,55% linhas

8.2 Como Reproduzir

```
# Compilação
cargo check -p server && cargo check -p app && cargo check -p frontend

# Testes (requer PostgreSQL)
DATABASE_URL=postgres://ben:1234@localhost/fuzzysimulated_test \
cargo test -p server -- --skip ignored

# Servidor de desenvolvimento
```

9. Conclusão

O FuzzySimulated demonstra a aplicação prática e completa de sistemas de controle fuzzy em dois domínios distintos: **Conforto Térmico** (avaliação ambiental) e **Risco Cibernético** (cibersegurança). A plataforma implementa os dois principais paradigmas de inferência (Mamdani e TSK), oferece ferramentas de validação (superfície, sweep, diagnóstico) e inclui otimização automática via PSO.

O sistema Conforto Térmico, com 2 entradas, 5 termos de saída e 17 regras (incluindo 8 regras extremas com MFs gaussianas), mostrou-se consistente na avaliação de 10 cenários climáticos, variando de desconfortável (frio/seco) a confortável (temperatura agradável/umidade normal). O sistema Risco Cibernético, com 3 entradas mapeadas do dataset_ml.parquet e 19 regras, demonstra a capacidade de processamento em lote e a integração com dados reais de cibersegurança.

Limitações

- O modelo não considera fatores humanos (treinamento, cultura de segurança)
- Não há aprendizado contínuo a partir de novos incidentes
- O PSO otimiza parâmetros off-line, não em tempo real

Trabalhos Futuros

- Integração com feeds de ameaças em tempo real (CVE, MITRE ATT&CK)
- Extensão Neuro-Fuzzy para aprendizado de parâmetros a partir de dados históricos
- Módulo de recomendação automática de contramedidas

10. Declaração de Uso de IA

Ferramenta	Finalidade	Prompt/Comando Resumido	Revisão Humana
DeepSeek V4 Flash (via opencode CLI)	Geração de engine fuzzy (membership, Mamdani, parser)	"Implemente motor Mamdani com trimf/trapmf/gaussmf e defuzz centroide"	Testes unitários validam cada função matemática; parâmetros validados contra NaN/Inf
DeepSeek V4 Flash (via opencode CLI)	Corrupt-params + PSO auto demo	"Crie endpoint que degenera MFs para demonstrar recuperação PSO"	Saída verificada: MSE ~5000 antes, ~988 depois
DeepSeek V4 Flash (via opencode CLI)	Batch parquet mapping	"Mapeie colunas do dataset_ml.parquet para variáveis fuzzy"	Bulk INSERT de 778 registros; saída normalizada [0,100]
DeepSeek V4 Flash (via opencode CLI)	Componentes Leptos (frontend)	"Crie página de simulação com abas Mamdani/TSK/SVG/Diagnóstico"	Interface validada visualmente; testes E2E com Playwright
DeepSeek V4 Flash (via opencode CLI)	Rotas Axum (backend)	"Crie rota CRUD para sistemas fuzzy com validação e auditoria"	Rotas testadas via HTTP Axum (64 testes)
DeepSeek V4 Flash (via opencode CLI)	Migrations SQL	"Crie migration seed Risco Cibernético com JSONB"	Migrations testadas com rollback; dados verificados no banco
DeepSeek V4 Flash (via opencode CLI)	Testes	"Crie testes HTTP para rota de variáveis"	is_ok() substituído por expect() para debug
DeepSeek V4 Flash (via opencode CLI)	Documentação	"Documente arquitetura do sistema"	Conteúdo revisado e ajustado conforme implementação real

Todas as sugestões de IA foram revisadas, testadas e validadas pelo integrante da equipe. O código gerado por IA passou por revisão manual e bateria de testes antes de ser integrado.

11. Referências

- Zadeh, L. A. (1965). *Fuzzy sets*. Information and Control, 8(3), 338–353.
- Mamdani, E. H., & Assilian, S. (1975). *An experiment in linguistic synthesis with a fuzzy logic controller*. International Journal of Man-Machine Studies, 7(1), 1–13.
- Takagi, T., & Sugeno, M. (1985). *Fuzzy identification of systems and its applications to modeling and control*. IEEE Transactions on Systems, Man, and Cybernetics, 15(1), 116–132.
- Kennedy, J., & Eberhart, R. (1995). *Particle swarm optimization*. Proceedings of IEEE International Conference on Neural Networks, 1942–1948.
- logicfuzzy_academic v0.2.1 — Crate Rust para lógica fuzzy acadêmica.
- Rust Programming Language — <https://www.rust-lang.org/>
- Leptos Framework — <https://leptos.dev/>
- Axum Web Framework — <https://github.com/tokio-rs/axum>
- SQLx — <https://github.com/launchbadge/sqlx>